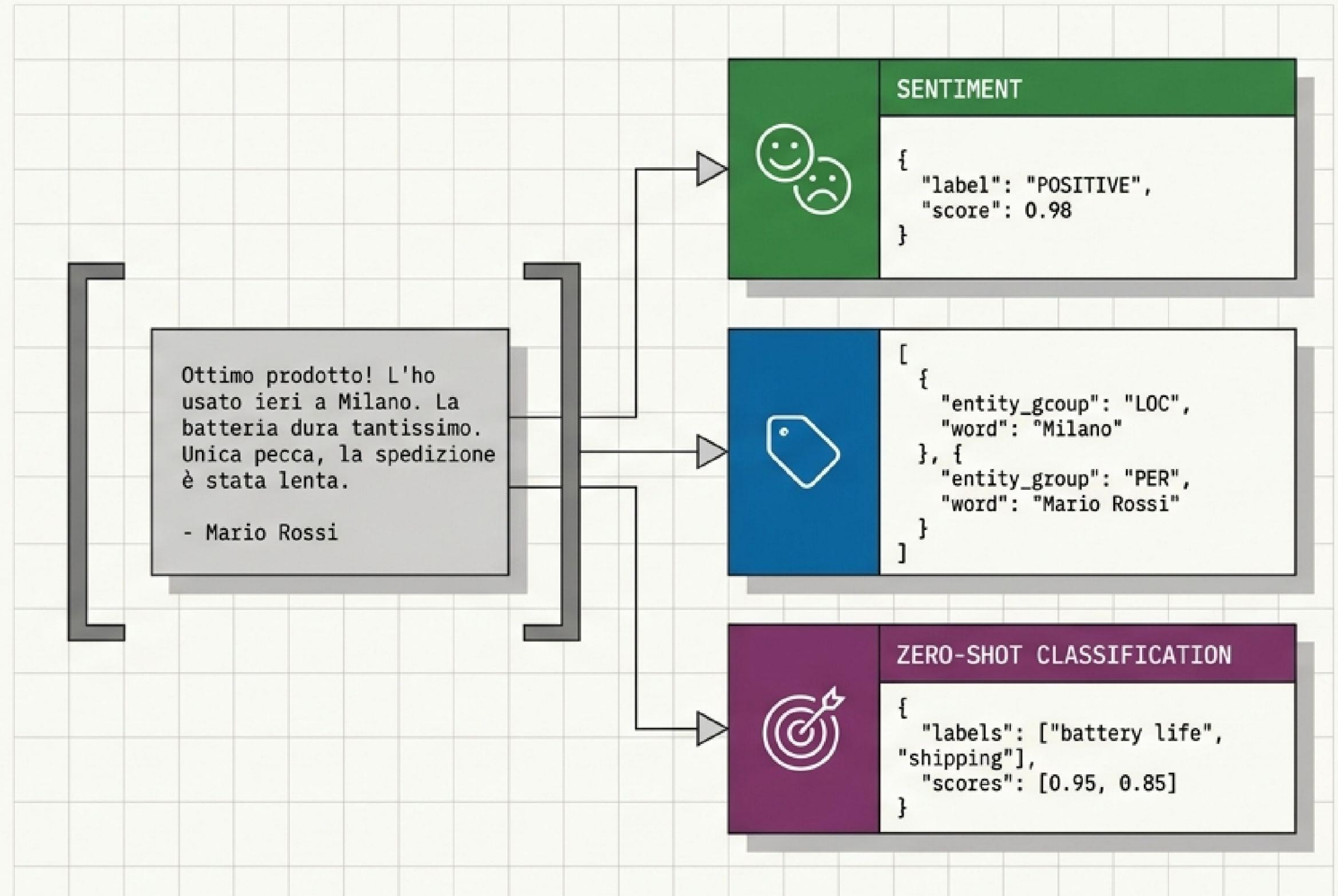


NLP & Generative AI con modelli open source

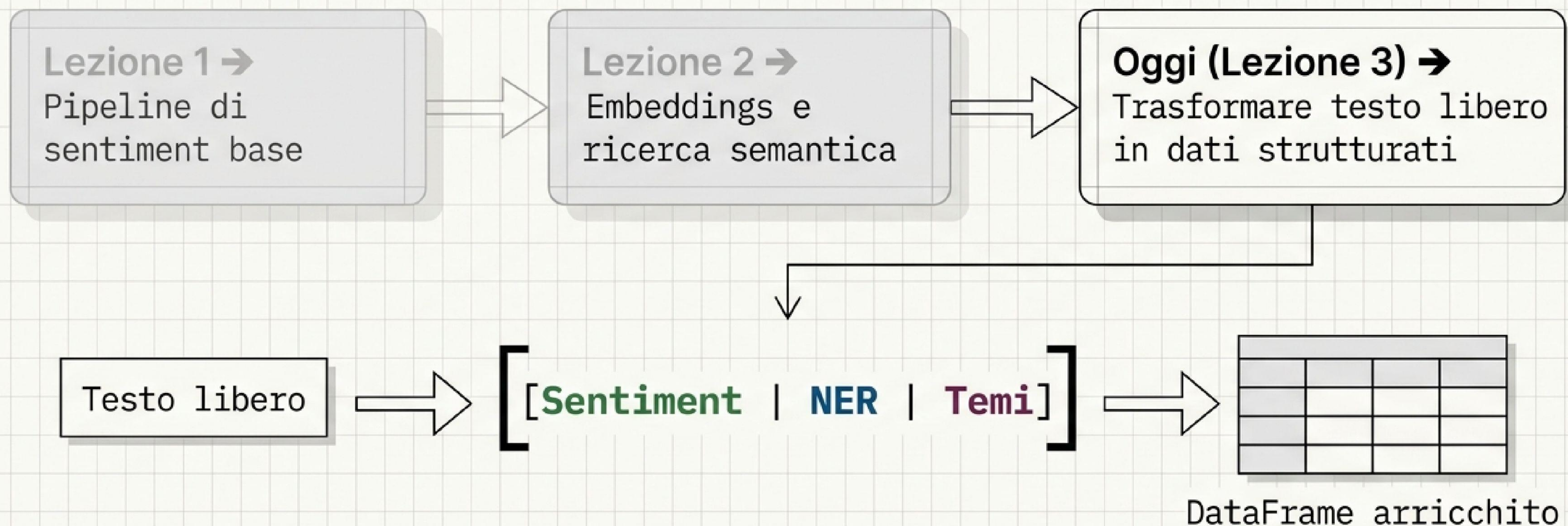
Lezione 3 – Modelli task-specific: sentiment, NER e classificazione

- >_ Estrarre informazioni strutturate dalle recensioni in linguaggio libero
- >_ Tre compiti, tre modelli specializzati, una riga di codice ciascuno
- >_ Ambiente:
Google Colab + GPU T4



Il percorso della pipeline: dove eravamo e dove andiamo

Obiettivo di oggi: estrarre sentiment, entità e temi per ogni recensione per costruire il dataset definitivo.

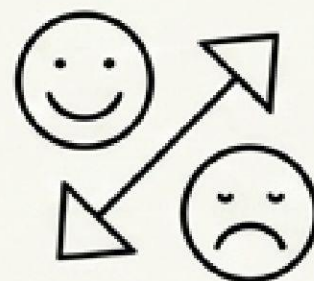


Obiettivi dell'architettura di oggi

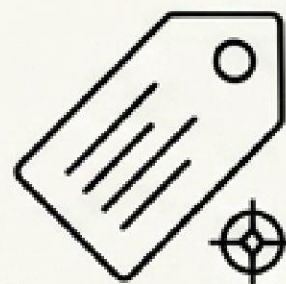
1. Capire il trade-off:
Task-specific
vs LLM
generativi.



2. Sentiment:
estrarre
polarità
(positivo/
negativo).



3. NER:
estrarre entità
(prodotti,
marchi,
luoghi).



4. Zero-shot:
classificare
temi senza
addestramento.

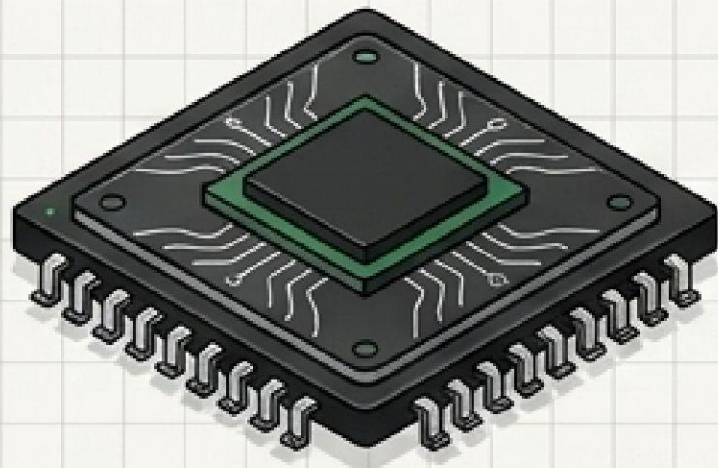


5. Costruire
l'arricchimento
riusato nel
progetto finale
(Lezione 6).



L'hardware detta l'architettura: Due famiglie di modelli

Modelli Task-specific



Dimensione
MB



Velocità

Compito: Uno solo

Costo HW: Basso

Output: Strutturato [{ }]

LLM Generativi



Dimensione
GB



Velocità

Compito: Molti

Costo HW: Alto

Output: Testo libero "..."

⇒ Compito ben definito e ripetuto su molti testi → usa un modello specializzato.

L'albero decisionale: Quando usare quale

Il task richiede di generare testo nuovo o ragionare in modo aperto?

No

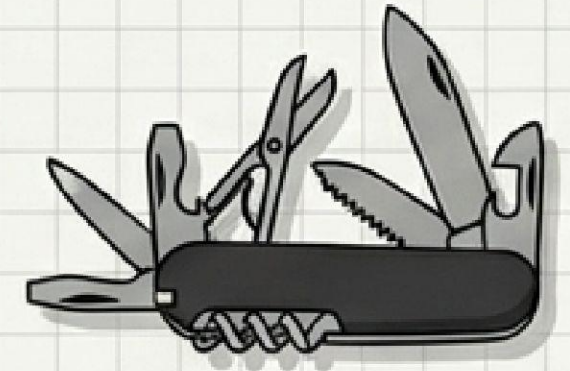
Sì



Modelli specializzati

- qual è il sentiment?
- quali entità?
- quale categoria?

Veloci, economici, scalano su migliaia di testi.



LLM generativi

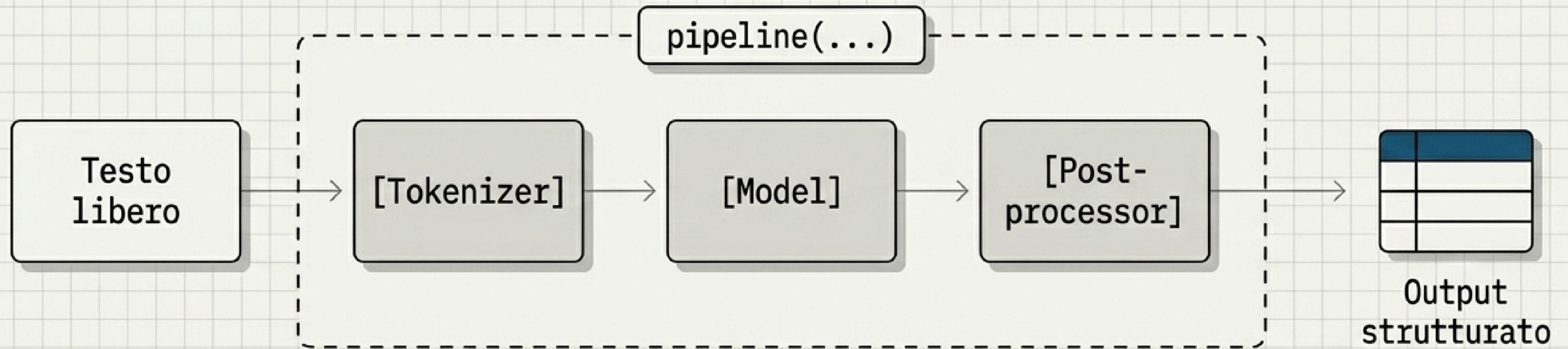
- riassumi
- rispondi
- riscrivi

Versatili, adatti a ragionamenti complessi.

Regola d'oro: Prima prova un modello task-specific.
Passa all'LLM solo se il compito richiede la generazione di testo.

L'astrazione: Anatomia di una pipeline Hugging Face

Una pipeline = modello + tokenizer + pre/post-processing, in una singola riga di codice.



text-classification
→ sentiment

token-classification
→ NER

zero-shot-classification
→ temi

Task 1: Pipeline di Sentiment 😊😐😡

Compito: Classificare in negativo / neutro / positivo

Modello: neuraly/bert-base-italian-cased-sentiment

```
classifier = pipeline('text-classification', model='...')
```

Applicato a tutte le recensioni → genera la nuova colonna sentiment.

Suono incredibile e batteria infinita.

[LABEL: positive] | score: 0.98

Materiali scadenti, si sono rotte subito.

[LABEL: negative] | score: 0.95

Task 2: Named Entity Recognition (NER)

Trova i 'nomi propri' nel testo e li classifica.

Ho comprato le Cuffie XSound ^[ORG] a Roma _[LOC] per Mario _[PER].

[PER] — persone

[ORG] — organizzazioni, aziende, marchi

[LOC] — luoghi

[MISC] — altre entità (prodotti, eventi...)

NER in pratica: La strategia di aggregazione

Without strategy



`aggregation_strategy='simple'`
ricuce i sub-token in entità intere
(senza, otterremmo i singoli pezzi di
parola).

```
pipeline('token-classification', aggregation_strategy='simple')
```

Per ogni entità → testo + tipo + score.
Genera la nuova colonna entita.

With aggregation_strategy

[Cuffie XSound: ORG]

Task 3: Classificazione Zero-Shot

Zero-shot = classificare senza addestramento, scegliendo le etichette al volo.
Le etichette sono semplici stringhe, decise da noi sul momento.



Modello: MoritzLaurer/mDeBERTa-v3-base-mnli-xnli

`multi_label=True`: una recensione può toccare più temi contemporaneamente.

Vantaggio Architettuale: Cambiare le etichette è gratis, nessun ri-addestramento richiesto.

Zero-Shot sulle recensioni: Estrarre il tema dominante

Array di Etichette: ['spedizione', 'prezzo', 'qualità del prodotto', 'assistenza clienti', 'facilità d'uso']

Le cuffie suonano incredibilmente bene, la profondità dei bassi è sbalorditiva.
La spedizione però è stata lenta e il prezzo è alto per quello che offrono.

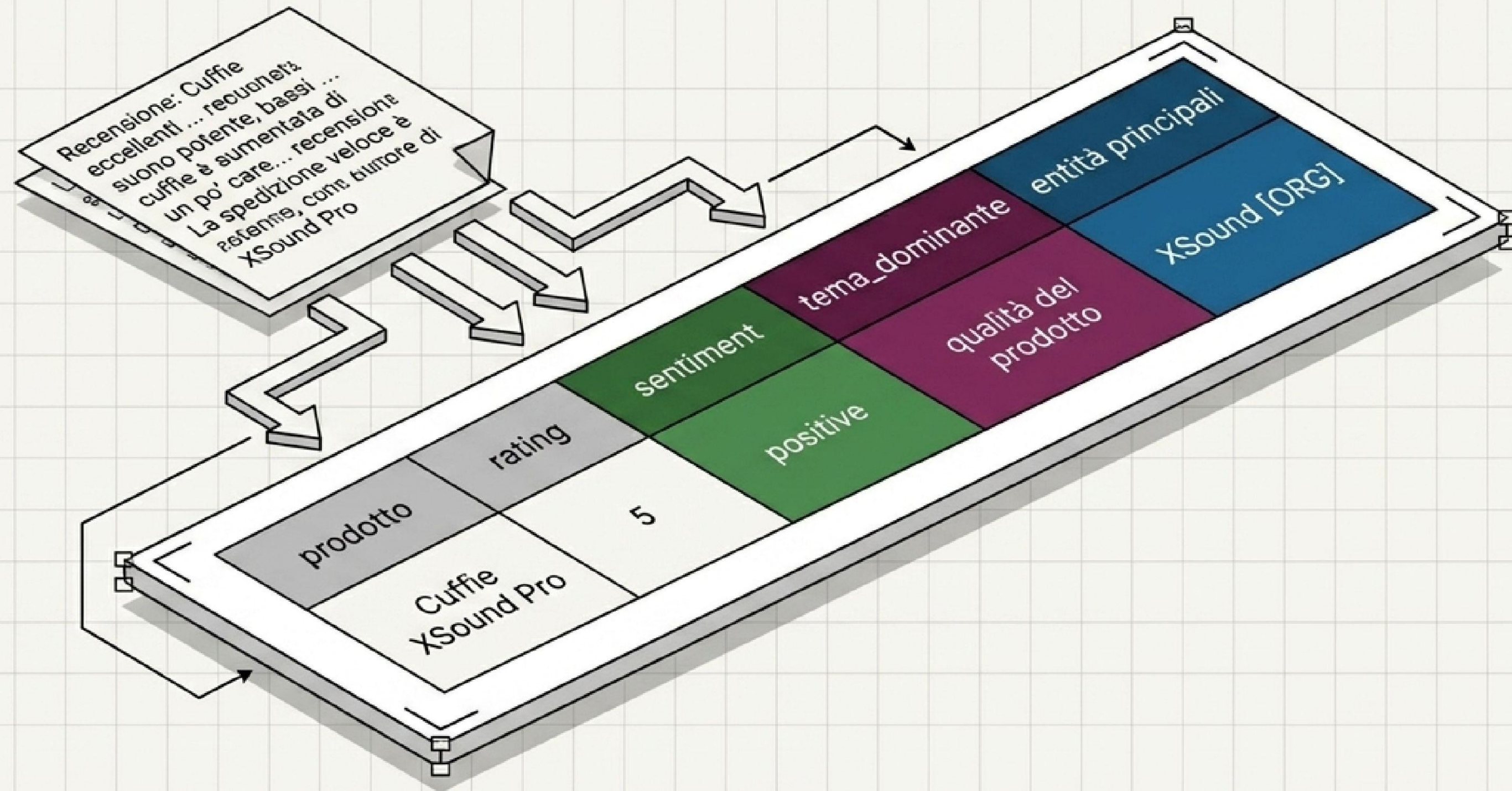


argmax() → Dominante

Processo: Per ogni recensione si calcola il punteggio per ogni tema.
Tramite l'operazione di argmax teniamo il tema dominante.

Output: Genera la nuova colonna tema_dominante.

Sintesi della Pipeline: Il DataFrame arricchito ✨



Da **testo libero** a **dati strutturati**. Questo dataset sarà l'input fondamentale nel progetto finale (Lezione 6) per alimentare il sistema **RAG** (Retrieval-Augmented Generation).

Laboratorio Pratico: Demo & Esercizio

Nel Notebook Colab: 

- Applichiamo i tre modelli (`Sentiment`, `NER`, `Zero-shot`) in batch a 200 recensioni.
- Costruiamo il `DataFrame` arricchito.

Sfida / Esercizio

Aggiungere una nuova etichetta custom `tempi di consegna` alla pipeline `Zero-Shot`, riclassificare il dataset, e visualizzare la distribuzione dei temi per rating tramite `pd.crosstab`.

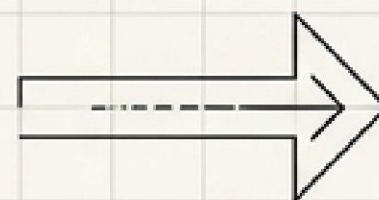
	1	2	3	4	5
qualità	4	10	11	5	15
spedizione	3	32	18	6	7
prezzo	0	15	20	8	7
assistenza clienti	1	7	27	9	5
facilità d'uso	0	4	27	13	17
tempi di consegna	0	0	2	1	16

Checkpoint architetturale e prossimi passi

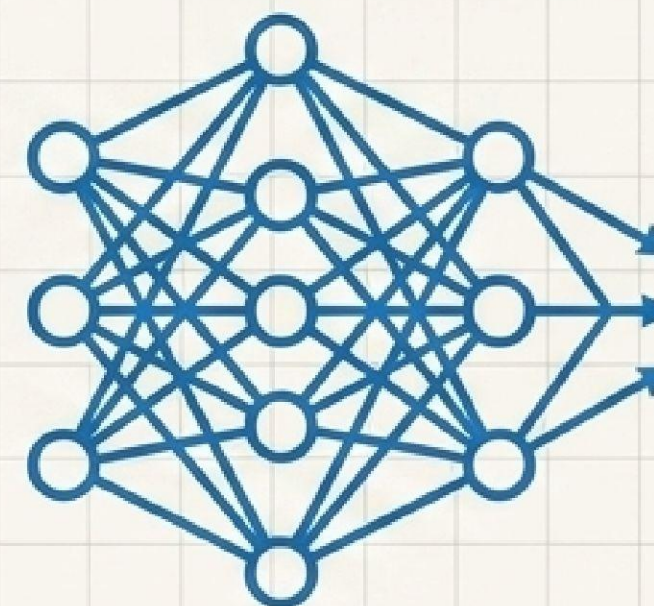
Oggi (Completato):



Tre modelli task-specific eseguiti. Pipeline consolidata. Recensioni da testo libero a dataset strutturato e analizzabile.



Lezione 4 (LLM Generativi):



Quando i task specifici non bastano e serve generare, riassumere o rispondere. Eseguiamo LLM open source direttamente in VRAM sulla GPU T4.